Implementation Design Document

SAANS

Symmetry-Safe Adaptive Noise Scheduling for E(3)-Equivariant Diffusion Training

Research-to-Implementation Blueprint for an EECE 571F Project

Purpose. This document converts the project idea into an implementation-ready technical plan. It is written to support four outcomes simultaneously:

1. a correct and reproducible baseline reproduction,
2. a principled implementation of the SAANS scheduler,
3. a rigorous experimental study suitable for a strong course project, and
4. a foundation that can be extended into a workshop or main-track research submission if empirical results are strong.

Version 1.0
March 8, 2026

Contents

1 Executive Summary **4**

2 Research Motivation and Project Scope **5**

2.1 What problem this project solves 5

2.2 Primary contribution 5

2.3 Scope boundaries 6

3 Technical Background and Notation **7**

3.1 Data representation 7

3.2 Baseline diffusion objective 7

3.3 Equivariance and invariance requirements 7

3.4 Bin-wise formulation for practical implementation 8

4 Formal SAANS Method Design **9**

4.1 Core adaptive objective 9

4.2 Hardness statistics 9

4.3 EMA update rule 10

4.4 Adaptive probability construction 10

4.5 Importance weights and stabilization 11

4.6 Reference training loop 11

4.7 Implementation invariants 12

5 System Architecture and Repository Layout **13**

5.1 Recommended repository structure 13

5.2 Module responsibilities . . . 14
5.2.1 Backbone adapter . . . 14
5.2.2 Scheduler subsystem . . . 14
5.3 Configuration design . . . 15

6 Implementation Phases **16**
6.1 Phase 0: Baseline selection and reproduction . . . 16
6.2 Phase 1: Instrument the timestep pipeline . . . 16
6.3 Phase 2: Implement the scheduler . . . 17
6.4 Phase 3: Correctness validation . . . 17
6.5 Phase 4: Toy sanity study . . . 18
6.6 Phase 5: Main QM9 study . . . 18
6.7 Phase 6: Harder benchmark or transfer validation . . . 19

7 Experimental Design **20**
7.1 Research hypotheses . . . 20
7.2 Datasets . . . 21
7.2.1 Primary dataset: QM9 . . . 21
7.2.2 Secondary dataset: GEOM-Drugs or equivalent . . . 21
7.3 Main baselines . . . 21
7.4 Primary metrics . . . 21
7.4.1 Generation quality . . . 21
7.4.2 Optimization efficiency . . . 22
7.4.3 Schedule diagnostics . . . 22
7.5 Statistical protocol . . . 22
7.6 Ablation matrix . . . 23

8 Diagnostics, Logging, and Visualization Plan **24**

9 Testing and Verification Strategy **25**
9.1 Unit tests . . . 25
9.2 Integration tests . . . 25

9.3 Failure signals to monitor early 26

10 Compute Plan and Resource Strategy **27**

10.1 Resource tiers 27

10.2 Compute allocation strategy 27

10.3 Reproducibility requirements 27

11 Project Timeline and Deliverables **29**

11.1 Suggested 8-week plan 29

11.2 Milestone deliverables 29

12 Risk Register and Mitigation **31**

13 Paper Framing Strategy **32**

14 Definition of Done **33**

A Implementation Checklist **34**

B Minimal Reporting Template **35**

Chapter 1

Executive Summary

The project studies whether diffusion training for 3D molecular generation can be made more efficient by *adaptively reallocating training effort across noise levels* without changing the original objective. The core method, SAANS, changes how timesteps are sampled during training while preserving the baseline target objective using importance weighting. The scheduler must also be *symmetry-safe*: difficulty estimates should not depend on arbitrary rotations or translations of a molecule.

This design document assumes the implementation will begin from an existing E(3)-equivariant molecular diffusion baseline, then add a modular timestep-sampling subsystem rather than changing the backbone architecture. The key engineering principle is to separate **model code**, **diffusion/noising code**, and **sampling policy code**, so that SAANS can be switched on and off cleanly for controlled experimentation.

The recommended execution path is:

1. reproduce a baseline on QM9,
2. instrument per-timestep loss and variance diagnostics,
3. implement bin-wise adaptive timestep sampling with objective-preserving reweighting,
4. validate correctness on toy problems and unit tests,
5. run the main QM9 ablation study,
6. extend to a harder dataset or setting if results justify it.

Chapter 2

Research Motivation and Project Scope

2.1 What problem this project solves

Diffusion models train by corrupting data with noise at different timesteps and asking a model to predict denoising targets. In practice, not all timesteps are equally informative or equally difficult. A fixed or hand-designed sampling schedule may overspend compute on easy regions of the noise space and undersample difficult ones.

For E(3)-equivariant molecular diffusion, this issue is especially interesting because:

- the denoising problem changes significantly across low-, mid-, and high-noise regimes,
- optimization may suffer from high variance if the training distribution is poorly allocated, and
- any adaptive strategy must respect 3D geometric symmetries.

The central question is therefore:

> Can we adaptively allocate training probability mass toward the currently difficult timestep regions in E(3)-equivariant diffusion, while preserving the original training objective and maintaining symmetry-consistent behavior?

2.2 Primary contribution

This project does **not** aim to invent a new molecular diffusion backbone. Instead, it contributes a **training-time scheduling mechanism** with three intended properties:

1. **Objective preservation:** adaptive timestep sampling should still optimize the same baseline objective in expectation.
2. **Symmetry safety:** hardness estimates used by the scheduler should be invariant to rigid geometric transformations when appropriate.
3. **Optimization relevance:** the method should reduce wasted compute, improve sample efficiency, or reduce optimization variance relative to static schedules.

2.3 Scope boundaries

To keep the project publishable-capable but still finishable, the initial scope should remain narrow:

- **In scope:** baseline reproduction, adaptive scheduler implementation, correctness validation, diagnostics, ablations, QM9 study, optional harder benchmark.
- **Out of scope initially:** changing the backbone network, proposing a new chemistry objective, building a new benchmark from scratch, or mixing multiple independent research ideas in the first version.

Chapter 3

Technical Background and Notation

3.1 Data representation

Let a molecular example be represented as

$$x = (\mathbf{r}, \mathbf{z}),$$

where $\mathbf{r} \in \mathbb{R}^{N\times 3}$ denotes atomic coordinates and $\mathbf{z}$ contains node features such as atom types or learned categorical embeddings. Depending on the selected baseline, the model may predict coordinate noise, score functions, denoised coordinates, or a combination of coordinate and feature outputs.

3.2 Baseline diffusion objective

Let t denote a diffusion timestep or continuous noise level. Baseline training samples $t \sim p_0(t)$ and optimizes

$$\mathcal{J}(\theta) = \mathbb{E}_{x\sim p_{\text{data}},\, t\sim p_0}\big[\mathcal{L}_\theta(x,t)\big].$$

The exact form of $\mathcal{L}_\theta$ depends on the backbone. The design in this document intentionally remains agnostic to whether the model is formulated in DDPM, score-based, or EDM-style notation. The only hard requirement is that the implementation can expose *per-example* or *per-bin* detached loss statistics to the scheduler.

3.3 Equivariance and invariance requirements

For molecular generation in 3D, rigid transformations of the coordinates should not alter the semantic identity of a sample. We therefore rely on the following principles:

- **Equivariance:** if the input coordinates are rotated, the model's vector-valued outputs should rotate in the corresponding way.
- **Invariance:** scalar diagnostics used by the scheduler should not depend on arbitrary orientation or translation.

This means the scheduler should be driven by invariant hardness summaries, not raw coordinate-frame-dependent heuristics.

3.4 Bin-wise formulation for practical implementation

A continuous adaptive density over t is possible in principle, but a **bin-wise formulation** is much easier to implement robustly in existing codebases. Partition the timestep domain into disjoint bins

$$I_1, I_2, \ldots, I_B.$$

Define baseline bin masses

$$p_{0,b} = \Pr_{t \sim p_0}(t \in I_b).$$

The adaptive scheduler will sample a bin b first, then draw t from the *baseline conditional distribution inside that bin.* This design has three major benefits:

1. it simplifies importance weights,
2. it prevents implementation drift away from the baseline time parameterization, and
3. it works in both discrete-time and continuous-time diffusion codebases.

Chapter 4

Formal SAANS Method Design

4.1 Core adaptive objective

Let the scheduler define an adaptive bin distribution q_b over bins. The adaptive sampling procedure is:

1. sample bin $b \sim q$,
2. sample $t \sim p_0(t \mid t \in I_b)$,
3. compute the model loss $\mathcal{L}_\theta(x, t)$,
4. weight the loss by
$$w_b = \frac{p_{0,b}}{q_b}.$$

Then the weighted objective estimator is

$$\tilde{\mathcal{J}}(\theta) = \mathbb{E}_{x \sim p_{\text{data}},\, b \sim q,\, t \sim p_0(\cdot \mid I_b)} \big[w_b \, \mathcal{L}_\theta(x, t) \big].$$

Because the conditional within each bin remains the baseline conditional, the ratio simplifies to a constant within each bin. This is one of the most implementation-friendly aspects of the method.

4.2 Hardness statistics

Each bin maintains a detached hardness estimate S_b. The scheduler should never update hardness using tensors that keep gradient history. All scheduler state must be maintained under `torch.no_grad()` or an equivalent non-differentiable update path.

Recommended hardness candidates are:

1. **Invariant coordinate hardness:**

$$h^{\text{coord}} = \frac{1}{N}\sum_{i=1}^{N}\|\hat{\epsilon}_i - \epsilon_i\|_2^2,$$

   where the aggregated norm is rotation-invariant.

2. **Feature hardness:** detached feature loss such as cross-entropy or feature MSE, depending on the baseline.

3. **Combined hardness:**

$$h^{\text{combo}} = \lambda_{\text{coord}} h^{\text{coord}} + \lambda_{\text{feat}} h^{\text{feat}}.$$

The initial implementation should start with one scalar hardness value per training sample, then aggregate by bin inside a minibatch.

4.3 EMA update rule

For each bin b, maintain an exponential moving average

$$S_b^{(n)} = \beta S_b^{(n-1)} + (1-\beta)\,\bar{h}_b^{(n)},$$

where $\bar{h}_b^{(n)}$ is the mean hardness among samples from bin b in iteration n, and $\beta \in [0,1)$ is a smoothing constant.

Design requirements for the EMA subsystem:

- missing bins in a minibatch should leave their EMA unchanged,
- all state updates must be numerically safe under mixed precision,
- the tracker should record both raw batch hardness and smoothed hardness,
- the implementation should support resetting, warm-starting, and checkpoint serialization.

4.4 Adaptive probability construction

Define positive bin scores using hardness values:

$$u_b = (\varepsilon + S_b)^{\alpha},$$

where $\alpha \geq 0$ controls how aggressively the scheduler concentrates on difficult bins and $\varepsilon > 0$ prevents degenerate zero scores.

The unregularized adaptive distribution is

$$\tilde{q}_b = \frac{p_{0,b} u_b}{\sum_{j=1}^{B} p_{0,j} u_j}.$$

To ensure full support and improve stability, mix with the baseline distribution:

$$q_b = (1-\rho)\tilde{q}_b + \rho p_{0,b}, \qquad 0 < \rho \leq 1.$$

This baseline mixing is mandatory in the reference implementation because it guarantees $q_b > 0$ whenever $p_{0,b} > 0$ and prevents catastrophic weight explosion.

4.5 Importance weights and stabilization

The default, theoretically clean implementation uses

$$w_b = \frac{p_{0,b}}{q_b}.$$

The design document recommends the following stabilization strategy:

1. **Warm start:** use the baseline schedule only for the first N_{warm} steps.
2. **Floor mixing:** enforce baseline mixing with parameter ρ as above.
3. **Monitoring before clipping:** log the full weight histogram first.
4. **Optional clipped variant:** if clipping is introduced, it must be treated as a separate ablation because it changes the estimator.

The unclipped weighted estimator should remain the primary reported method. A clipped variant may be useful as an engineering fallback but should not replace the main experiment unless numerical instability makes it unavoidable.

4.6 Reference training loop

The intended training procedure is:

```
for step in range(num_steps):
    batch = next(data_loader)

    if step < warmup_steps:
        t, bin_idx, q = baseline_sampler.sample(batch_size)
```

```
    else:
        t, bin_idx, q = saans_sampler.sample(batch_size)

    noisy_batch, targets = diffuser.corrupt(batch, t)
    outputs = model(noisy_batch, t)

    per_example_loss, loss_parts = loss_fn(outputs, targets, return_components=True)
    weights = sampler.importance_weights(bin_idx)
    weighted_loss = (weights * per_example_loss).mean()

    optimizer.zero_grad(set_to_none=True)
    weighted_loss.backward()
    optimizer.step()

    with no_grad():
        hardness = hardness_builder(loss_parts, outputs, targets)
        sampler.update(bin_idx, hardness)
        logger.log(step, q, weights, per_example_loss, weighted_loss, hardness)
```

4.7 Implementation invariants

The following properties must hold in the final system:

1. setting $\alpha = 0$ must recover the baseline schedule exactly,
2. setting $\rho = 1$ must recover the baseline schedule exactly,
3. the support condition $q_b > 0$ for all bins with $p_{0,b} > 0$ must always hold,
4. scheduler updates must be detached from the computation graph,
5. checkpoint restore must reproduce the scheduler state exactly,
6. metric logging must include enough information to reconstruct schedule evolution after training.

Chapter 5

System Architecture and Repository Layout

5.1 Recommended repository structure

Even if the baseline codebase already exists, the SAANS-specific additions should be isolated cleanly. A recommended structure is:

```
docs/
  saans_design_document.tex

configs/
  qm9_baseline.yaml
  qm9_saans.yaml
  geom_saans.yaml

src/
  data/
    qm9.py
    geom_drugs.py
  models/
    backbone_adapter.py
  diffusion/
    noising.py
    losses.py
  scheduler/
    bins.py
    baseline_sampler.py
    hardness.py
    tracker.py
    saans_sampler.py
```

```
  training/
    trainer.py
    hooks.py
    checkpointing.py
  eval/
    generation.py
    molecule_metrics.py
    optimization_metrics.py
  utils/
    logging.py
    seeding.py
    geometry.py

tests/
  test_sampler_equivalence.py
  test_importance_weighting.py
  test_hardness_invariance.py
  test_checkpoint_resume.py

scripts/
  run_qm9_baseline.sh
  run_qm9_saans.sh
  run_toy_sanity.sh
```

5.2 Module responsibilities

5.2.1 Backbone adapter

Because the baseline may come from an external repository, a small adapter layer should expose a stable internal API:

- `forward(noisy_batch, t)`
- `loss_fn(outputs, targets, return_components=True)`
- `sample(...)`

The goal is to prevent SAANS logic from depending on the exact internal conventions of the chosen diffusion implementation.

5.2.2 Scheduler subsystem

The scheduler subsystem should consist of four separate concepts:

1. **Bin definition:** maps timesteps to bins and stores $p_{0,b}$.
2. **Hardness builder:** converts detached loss components into invariant scalar hardness values.
3. **Tracker:** maintains EMA statistics and exposes smoothed difficulty estimates.
4. **Sampler:** converts tracked hardness into a valid adaptive distribution and returns importance weights.

Keeping these components separate is critical for ablations. For example, hardness variants should not require editing the sampler itself.

5.3 Configuration design

The project should be fully config-driven. Suggested configuration fields include:

```
scheduler:
  enabled: true
  num_bins: 32
  alpha: 1.0
  ema_beta: 0.95
  baseline_mix_rho: 0.1
  warmup_steps: 5000
  hardness_type: coord_plus_feat
  lambda_coord: 1.0
  lambda_feat: 1.0
  clip_weights: false
  max_weight: null
  log_every: 100
```

Other important configuration groups are dataset version, model backbone, training seed, precision mode, optimizer, and evaluation cadence.

Chapter 6

Implementation Phases

6.1 Phase 0: Baseline selection and reproduction

The first engineering decision is to choose one reliable E(3)-equivariant molecular diffusion baseline. The project should prefer a codebase that already has:

- stable training scripts,
- generation and evaluation utilities,
- reproducible QM9 support,
- a community-understood training objective.

Exit criterion: train and evaluate the baseline on QM9 with results that are at least directionally consistent with the original implementation or public reproductions.

6.2 Phase 1: Instrument the timestep pipeline

Before changing sampling behavior, identify and log the current timestep mechanics:

1. where timesteps are sampled,
2. whether they are discrete, continuous, or transformed internally,
3. how noise is injected,
4. how the loss depends on timestep,
5. what per-sample losses are accessible.

New instrumentation should log:

- sampled timestep histogram,
- mean and variance of loss per bin,
- gradient norm statistics if affordable,
- time-to-step and memory overhead.

Exit criterion: produce plots showing that different timestep regions exhibit different loss or variance behavior.

6.3 Phase 2: Implement the scheduler

This phase adds the binning, hardness tracking, adaptive probability computation, and importance weighting. It should be done incrementally:

1. implement static bins and baseline bin masses,
2. implement a baseline-compatible sampler using those bins,
3. implement detached hardness aggregation,
4. implement EMA tracking,
5. implement adaptive q_b generation,
6. integrate weighted loss computation,
7. add logging and checkpointing.

Exit criterion: on a smoke test, training runs end-to-end with SAANS enabled and all diagnostics logged.

6.4 Phase 3: Correctness validation

Correctness validation is not optional. The project's credibility depends on showing that the adaptive sampler is not just a heuristic hack.

Required tests:

- **Estimator equivalence test:** on a synthetic loss function with known baseline expectation, verify that weighted adaptive estimates match the baseline expectation within Monte Carlo error.
- **Support test:** verify $q_b > 0$ whenever $p_{0,b} > 0$.
- **Recovery test:** verify that $\alpha = 0$ or $\rho = 1$ recovers the baseline.
- **Invariance test:** rotate a batch rigidly and confirm the hardness statistic is unchanged up to numerical tolerance.
- **Resume test:** save and reload a checkpoint and confirm the scheduler state matches bitwise or near-bitwise.

Exit criterion: all sampler tests pass before expensive experiments begin.

6.5 Phase 4: Toy sanity study

Before burning substantial GPU time, run a toy diffusion setting where the effect of adaptive sampling is easy to visualize. The toy study should answer two questions:

1. does adaptive allocation change the sampled timestep distribution in the intended way?
2. under equal compute, does weighted adaptive sampling reduce optimization variance or reach a target loss faster?

Exit criterion: produce at least one figure showing schedule evolution and one figure showing convergence behavior.

6.6 Phase 5: Main QM9 study

This is the primary course-project experiment set. At minimum, run:

- baseline schedule,
- uniform or log-uniform alternative if supported,
- SAANS full method,
- adaptive without importance weighting,
- hardness ablation variants.

Exit criterion: obtain statistically credible trends across multiple seeds on QM9.

6.7 Phase 6: Harder benchmark or transfer validation

If results on QM9 are encouraging, run one stronger generalization study. The best candidate is a harder molecular dataset such as GEOM-Drugs, or a reduced subset of it if compute is limited.

Exit criterion: show whether the method still helps in a more challenging regime. This stage is highly valuable for publication potential.

Chapter 7

Experimental Design

7.1 Research hypotheses

The experiments should be designed around explicit hypotheses rather than vague benchmarking.

H1:
Different timestep regions contribute different levels of difficulty and optimization variance during equivariant diffusion training.

H2:
A symmetry-safe adaptive sampler can improve compute allocation without changing the underlying objective in expectation.

H3:
SAANS improves one or more of the following relative to static schedules: convergence speed, steps-to-target, wall-clock-to-target, gradient variance, or downstream generation quality.

H4:
Importance weighting is necessary; removing it changes the objective and leads to inferior or less reliable behavior.

H5:
Symmetry-safe hardness signals are more stable than naive non-invariant heuristics.

7.2 Datasets

7.2.1 Primary dataset: QM9

QM9 is the correct first benchmark because it is standard, comparatively small, and well-supported in many molecular generation codebases. It is the primary target for the course project and the right environment for debugging the adaptive scheduler.

7.2.2 Secondary dataset: GEOM-Drugs or equivalent

This stage is optional for course completion but strongly recommended for publication ambitions. Even a reduced but credible harder-benchmark study substantially strengthens the claim that the method is not a QM9-only optimization trick.

7.3 Main baselines

The design should compare against the strongest baselines available within the selected codebase and compute budget.

1. **Baseline scheduler:** original training schedule used by the reference implementation.
2. **Uniform or log-uniform scheduler:** if the diffusion codebase supports alternative hand-designed schedules.
3. **SAANS:** full method with weighting and symmetry-safe hardness.
4. **Adaptive-no-weighting:** adaptive scheduler without importance correction.
5. **Hardness variants:** coordinate-only, feature-only, and combined hardness.

7.4 Primary metrics

The exact molecule-quality metrics depend on the baseline repository, but the evaluation suite should include three categories.

7.4.1 Generation quality

Use the baseline's standard molecular generation metrics wherever possible, such as validity, uniqueness, novelty, or geometry-related quality scores. Reusing the baseline's own evaluation protocol is important for fair comparison.

7.4.2 Optimization efficiency

These metrics are central to the project and should be emphasized in the paper:

- steps-to-target metric threshold,
- wall-clock time to target,
- effective sample size of importance weights,
- gradient variance or proxy variance across training,
- area under the training-quality curve.

7.4.3 Schedule diagnostics

These are essential for explaining *why* the method works:

- evolution of q_b over training,
- hardness per bin over training,
- weight histogram over training,
- bin occupancy histogram,
- per-bin loss variance.

7.5 Statistical protocol

To avoid overclaiming from noisy runs, the study should follow a simple but rigorous protocol:

- use multiple random seeds for every main comparison,
- keep evaluation cadence identical across methods,
- report mean and standard deviation,
- use matched compute budgets when comparing methods,
- keep the backbone, optimizer, and dataset split fixed across scheduler variants.

7.6 Ablation matrix

The minimum meaningful ablations are shown in Table 7.1.

Table 7.1: Recommended ablation matrix for the initial study.

Ablation	**Purpose**
No weighting	Tests whether importance correction is necessary.
α sweep	Measures sensitivity to scheduler aggressiveness.
ρ sweep	Measures stability / support trade-off from baseline mixing.
Number of bins B	Tests discretization sensitivity.
Hardness type	Tests whether symmetry-safe signal choice matters.
Warmup length	Tests dependence on early-training schedule lock-in.
Optional weight clipping	Evaluates stability-bias trade-off if needed.

Chapter 8

Diagnostics, Logging, and Visualization Plan

The project will be much stronger if diagnostics are treated as first-class outputs rather than debugging leftovers. The following plots should be intentionally designed from the start:

1. timestep/bin occupancy over time,
2. EMA hardness trajectories per bin,
3. adaptive distribution heatmap over training,
4. importance weight histogram and effective sample size,
5. per-bin loss mean and variance,
6. training loss vs. wall-clock and vs. steps,
7. target metric attainment curves,
8. final molecule quality comparison table.

Recommended logging backends are lightweight experiment trackers such as TensorBoard, CSV logs, or Weights & Biases if available. The critical design rule is that every plotted quantity should also be stored in a machine-readable format for later paper figure generation.

Chapter 9

Testing and Verification Strategy

9.1 Unit tests

The test suite should not only check software behavior but also mathematical properties.

Table 9.1: Core unit tests for the SAANS implementation.

Test	**What it verifies**
Sampler normalization	The returned adaptive distribution sums to one and has no negative entries.
Baseline recovery	SAANS collapses to the baseline schedule under recovery settings.
Importance estimator	Weighted adaptive estimates recover known expectations in synthetic settings.
Bin mapping consistency	Timesteps map to the same bins during sampling and logging.
Hardness invariance	Hardness summaries remain unchanged under rigid transformations.
Checkpoint resume	Tracker state, bin masses, and config are restored exactly.
Mixed-precision safety	No NaNs or infs arise in sampler statistics under AMP.

9.2 Integration tests

Integration tests should run on tiny settings and confirm:

- end-to-end training does not crash,
- weighted loss backpropagates correctly,

- scheduler state updates after each step,
- metrics and diagnostics are logged as expected,
- resume-from-checkpoint training matches a continuous run closely.

9.3 Failure signals to monitor early

The following symptoms usually indicate bugs or design instability:

- one or more bins collapse to near-zero occupancy unexpectedly,
- importance weights explode or effective sample size collapses,
- schedule oscillates wildly between bins,
- hardness changes when the same molecule is rotated,
- SAANS appears to help only because the objective changed after removing weights,
- the overhead of the scheduler outweighs any training benefit.

Chapter 10

Compute Plan and Resource Strategy

10.1 Resource tiers

The implementation should be planned against three resource tiers:

Minimal tier:
one modern GPU, enough for smoke tests, toy studies, and limited QM9 experiments.

Recommended tier:
one to two stronger GPUs or access to a cluster, enough for multi-seed QM9 experiments and selected ablations.

Publication-capable tier:
additional cluster access for harder benchmarks, larger sweeps, or more extensive seed counts.

10.2 Compute allocation strategy

The highest-value compute path is:

1. use the smallest possible runs for correctness tests,
2. use moderate runs for scheduler hyperparameter selection,
3. reserve long runs for final baseline-vs-SAANS comparisons,
4. only after main QM9 evidence is positive, spend compute on the harder benchmark.

10.3 Reproducibility requirements

Every serious run should log:

- git commit hash,
- exact config file,
- random seed,
- hardware summary,
- software versions,
- checkpoint path,
- dataset split identifier.

Chapter 11

Project Timeline and Deliverables

11.1 Suggested 8-week plan

Table 11.1: Recommended execution schedule.

Week	Target outcome
1	Choose baseline, set up environment, reproduce at least a smoke-test training run.
2	Reproduce baseline metrics on QM9 and instrument timestep diagnostics.
3	Implement bins, tracker, and baseline-compatible sampler; add unit tests.
4	Integrate full SAANS weighting and logging; run toy sanity study.
5	Run main QM9 baseline-vs-SAANS comparisons with two to three seeds.
6	Run ablations on hardness, α, ρ, and warmup length.
7	Produce final tables/plots and decide whether harder-benchmark extension is justified.
8	Write report, polish figures, package code, and prepare presentation.

11.2 Milestone deliverables

The design should target the following concrete artifacts:

1. a reproducible baseline training script,
2. a scheduler module with unit tests,
3. a toy-sanity notebook or script,
4. a QM9 results folder with logs and plots,
5. a final report with method, diagnostics, and ablations,

6. a cleaned code release for submission or public sharing.

Chapter 12

Risk Register and Mitigation

Table 12.1: Primary project risks and mitigation plan.

Risk	**Why it matters**	**Mitigation**
Baseline reproduction fails	Project cannot progress credibly without a stable starting point.	Choose a well-documented baseline first; verify reproduction before any SAANS code is written.
Adaptive schedule is unstable	Large weight variance can erase benefits.	Use warmup, baseline mixing, close monitoring of weight histograms, and conservative α.
Hardness metric is poorly chosen	Scheduler may chase noise instead of true difficulty.	Start with simple invariant losses; compare coordinate-only and combined variants.
No benefit on QM9	Publication potential weakens.	Emphasize optimization diagnostics; test harder settings; avoid overfitting claims to one metric.
Too many ablations	Project becomes broad but shallow.	Lock the minimum ablation set early and defer extras until main evidence exists.
Compute bottleneck	Harder benchmark may become infeasible.	Prioritize QM9 correctness and quality first; use reduced harder benchmark only if justified.

Chapter 13

Paper Framing Strategy

If the empirical story is strong, the paper should be framed as an optimization-and-training paper for geometric generative models rather than as a new molecular architecture. A strong paper narrative is:

1. fixed timestep schedules waste training effort in equivariant diffusion,
2. naive adaptation changes the objective or ignores symmetry concerns,
3. SAANS provides a principled, symmetry-safe adaptive allocation mechanism,
4. diagnostics show how the scheduler reallocates effort,
5. experiments show better compute efficiency and competitive or improved generation quality.

The diagnostic story is almost as important as the final metric table. If the paper can show *why* the method helps, not just *that* it helps, the contribution becomes much more compelling.

Chapter 14

Definition of Done

The project should be considered complete only when all of the following are true:

- the baseline is reproduced and documented,
- SAANS is implemented as a clean, modular subsystem,
- correctness tests pass,
- the main QM9 study is completed with multiple seeds,
- at least the minimum ablation matrix is run,
- all plots needed for the report are generated from logged data,
- the final report can defend both the mathematical and engineering design decisions.

Appendix A

Implementation Checklist

1. Select baseline repository and freeze commit hash.
2. Create config files for baseline and SAANS.
3. Add timestep/bin instrumentation before changing behavior.
4. Implement bin manager and baseline bin masses.
5. Implement hardness builder with invariant reductions.
6. Implement EMA tracker and serialization.
7. Implement adaptive distribution and importance weights.
8. Integrate weighted loss into the training loop.
9. Add tests for estimator correctness and invariance.
10. Run toy study, then QM9 study, then harder extension if justified.

Appendix B

Minimal Reporting Template

When writing the final report or paper, the results section should be able to answer the following questions directly:

1. Which timesteps were hardest under the baseline?
2. How did SAANS reallocate sampling mass during training?
3. Did weighted adaptation improve efficiency under equal compute?
4. Did molecule quality improve, remain stable, or regress?
5. Was importance weighting necessary?
6. Did the symmetry-safe hardness design matter?
7. Did the result generalize beyond QM9?